

Mini-project 3 in Advanced Machine Learning (02460)

Zhenlin Xie (s232268), Ignacio Ripoll González (s242875), Alexander Hellberg Yanci (s253009), Eugene Ratkovich (s257470)

Model

The **encoder** applies $K=3$ message-passing (MP) layers to the node features $\mathbf{x}_v \in \mathbb{R}^7$, producing node embeddings $\mathbf{h}_v \in \mathbb{R}^{64}$ with residual connections. A sum-pooling readout gives a graph embedding $\mathbf{h}_G$, from which two linear heads produce the posterior parameters $\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2 \in \mathbb{R}^{32}$.

The **decoder** receives a latent sample $\mathbf{z} \sim q(\mathbf{z}|G) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$ concatenated with a normalised node count and passes it through an MLP to produce upper-triangular logits, from which independent Bernoulli edge probabilities are read off. The adjacency is symmetrised and self-loops removed.

The **training objective** is the ELBO with β -weighted KL divergence:

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{z}|G)}[\log p(A|\mathbf{z})] - \beta \text{KL}(q(\mathbf{z}|G) \| p(\mathbf{z})),$$

where $p(A|\mathbf{z})$ factorises over edges as independent Bernoulli and $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Training used Adam ($\eta=10^{-3}$, $\beta=1.0$, batch 32) with early stopping (patience 30); the model converged in 79 epochs.

Baseline

For each generated graph the Erdős-Rényi baseline (i) samples N from the empirical node-count distribution

of the training graphs, (ii) computes the link probability r as the mean density of training graphs with exactly N nodes, and (iii) samples $G(N, r)$.

Evaluation

Novelty and uniqueness are computed by hashing all graphs with the Weisfeiler-Lehman (WL) algorithm (`networkx.weisfeiler_lehman_graph_hash`) and resolving collisions with exact isomorphism checks (`networkx.is_isomorphic`).

	Novel	Unique	Novel+Unique
ER baseline	100.0%	99.7%	99.7%
Deep model	100.0%	100.0%	100.0%

Both models are fully novel; the deep model is also perfectly diverse. Figure 1 compares degree, clustering coefficient, and eigenvector centrality across the training set, ER baseline, and deep model using shared histogram bins. The deep model's degree distribution is closer to the training data than the ER baseline's, though pairwise KS tests confirm statistically significant differences in all three statistics for both models vs. training.

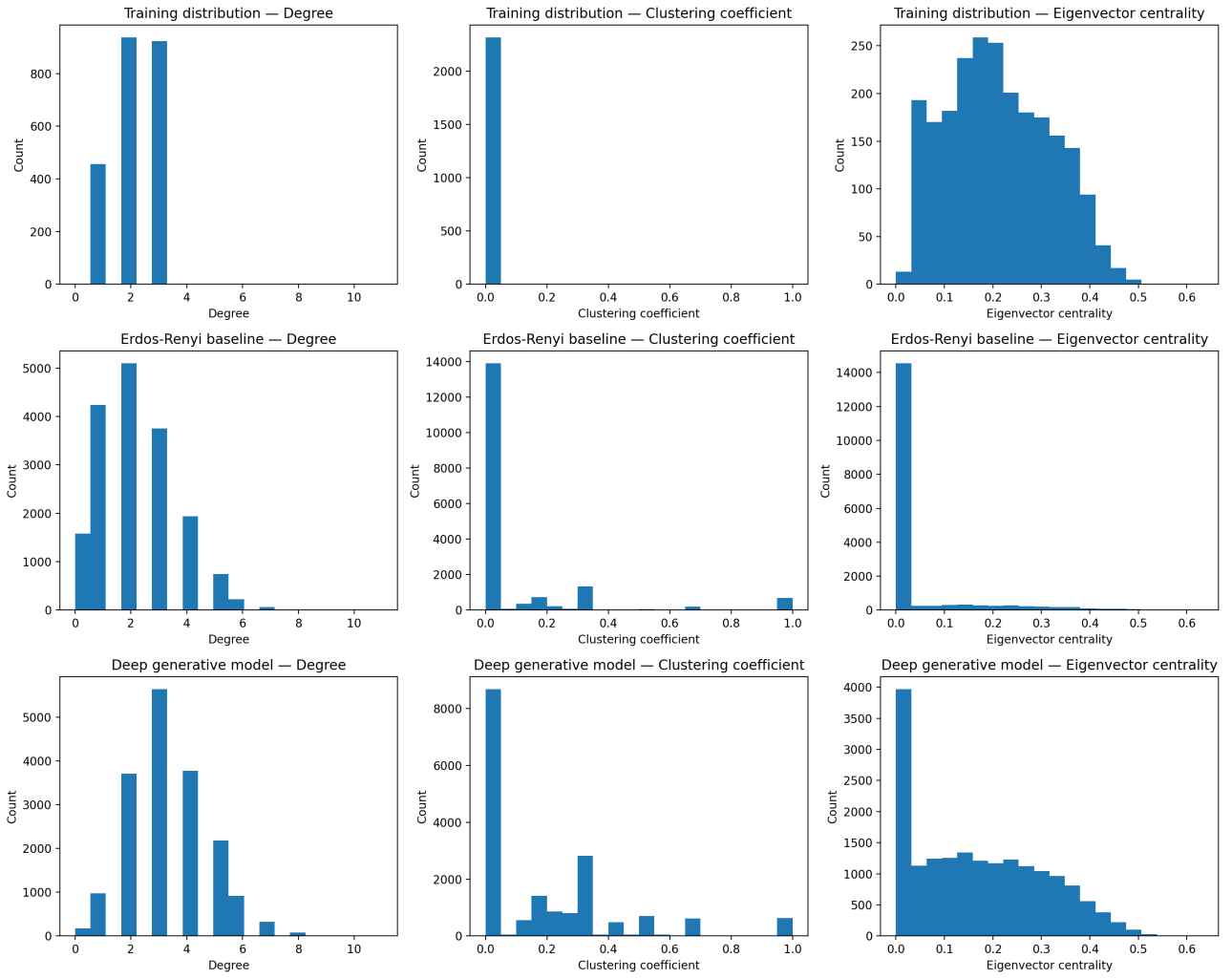


Figure 1: Histogram comparison (shared bins per statistic) of node degree, clustering coefficient, and eigenvector centrality for training data, ER baseline, and deep-model samples.

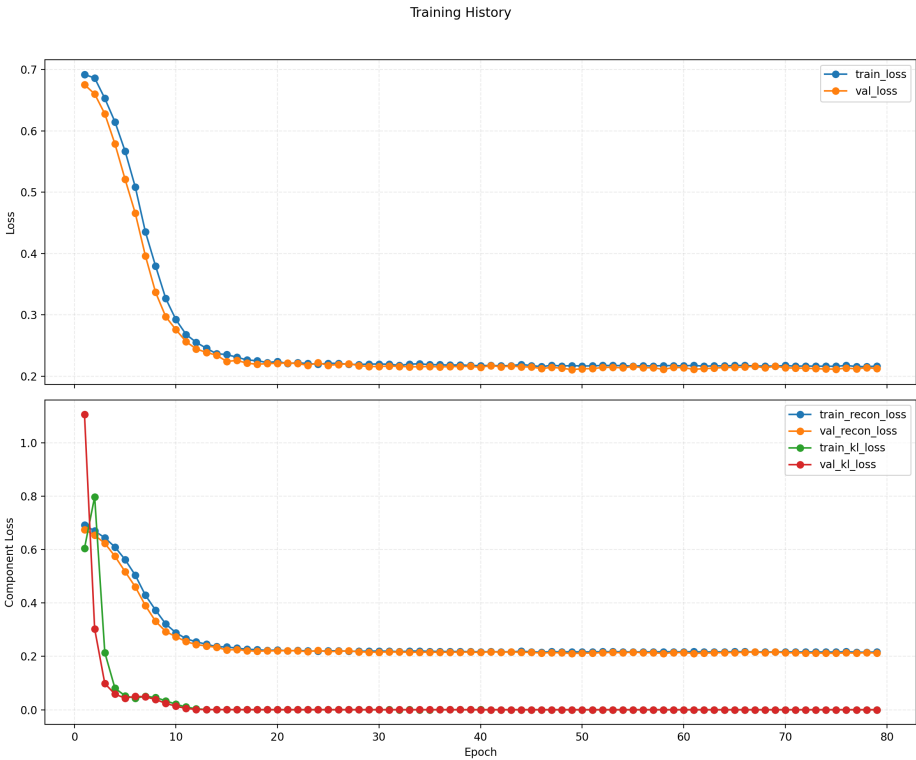


Figure 2: Training history: total loss (top) and reconstruction / KL components (bottom) over 79 epochs.

References

- [1] T. N. Kipf and M. Welling, “Variational Graph Auto-Encoders,” *NIPS Workshop on Bayesian Deep Learning*, 2016.
- [2] J. Gilmer, S. S. Schütt, G. E. Hinton, M. Welling, and D. Dürr, “Neural Message Passing for Quantum Chemistry,” *Proceedings of ICML*, 2017.
- [3] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, “Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds,” *Journal of Medicinal Chemistry*, 34(2):786–797, 1991.
- [4] C. Morris et al., “TUDataset: A collection of benchmark datasets for learning with graphs,” *ICML Workshop on Graph Representation Learning*, 2020.
- [5] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, “Weisfeiler-Lehman Graph Kernels,” *Journal of Machine Learning Research*, 12:2539–2561, 2011.

Code Snippets

```

1 class GraphEncoder(nn.Module):
2     def __init__(self, cfg: GraphVAEConfig):
3         super().__init__()
4         self.input_proj = nn.Linear(cfg.num_node_features, cfg.hidden_dim)
5         self.mp_layers = nn.ModuleList([
6             GCNConv(cfg.hidden_dim, cfg.hidden_dim)
7             for _ in range(cfg.num_message_passing_steps)
8         ])
9         self.mu_head = nn.Linear(cfg.hidden_dim, cfg.latent_dim)
10        self.logvar_head = nn.Linear(cfg.hidden_dim, cfg.latent_dim)
11
12    def forward(self, x, edge_index, batch):
13        h = F.relu(self.input_proj(x))
14        for layer in self.mp_layers:
15            h = F.relu(layer(h, edge_index)) + h # residual
16        h_G = global_add_pool(h, batch) # sum-pooling
17        return self.mu_head(h_G), self.logvar_head(h_G)

```

Listing 1: Message-passing encoder with sum-pooling readout (model_gvae.py).

```

1 class GraphDecoder(nn.Module):
2     def __init__(self, cfg: GraphVAEConfig):
3         super().__init__()
4         in_dim = cfg.latent_dim + (1 if cfg.use_size_conditioning else 0)
5         out_dim = cfg.max_nodes * (cfg.max_nodes - 1) // 2
6         self.mlp = nn.Sequential(
7             nn.Linear(in_dim, cfg.hidden_dim), nn.ReLU(),
8             nn.Linear(cfg.hidden_dim, cfg.hidden_dim), nn.ReLU(),
9             nn.Linear(cfg.hidden_dim, out_dim),
10        )
11
12    def forward(self, z, num_nodes=None):
13        if num_nodes is not None:
14            z = torch.cat([z, num_nodes.unsqueeze(-1)], dim=-1)
15        return torch.sigmoid(self.mlp(z)) # edge probabilities

```

Listing 2: MLP decoder: latent code to upper-triangular adjacency logits (model_gvae.py).

```

1 def elbo_loss(recon_probs, target_adj, mu, logvar, beta=1.0):
2     # Binary cross-entropy over upper-triangle edges
3     recon = F.binary_cross_entropy(recon_probs, target_adj, reduction='mean')
4     # KL divergence: -0.5 * sum(1 + logvar - mu^2 - exp(logvar))
5     kl = -0.5 * torch.mean(1 + logvar - mu.pow(2) - logvar.exp())
6     return recon + beta * kl, recon, kl

```

Listing 3: ELBO loss with beta-weighted KL (train_gvae.py).

```

1 def compute_novelty_metrics(generated_graphs, train_graphs):
2     train_buckets = build_hash_buckets(train_graphs) # WL hash -> [Graph]
3     unique_idx = set(unique_graph_indices(generated_graphs))
4     novel_flags = [is_graph_novel(g, train_buckets) for g in generated_graphs]
5     n = len(generated_graphs)
6     num_novel = sum(novel_flags)
7     num_unique = len(unique_idx)
8     num_nu = sum(1 for i, nov in enumerate(novel_flags) if nov and i in unique_idx)
9     return NoveltyMetrics(n, num_unique, num_novel, num_nu,
10        num_unique/n, num_novel/n, num_nu/n)
11
12 def build_hash_buckets(graphs):
13     buckets = {}
14     for g in graphs:
15         h = nx.weisfeiler_lehman_graph_hash(g) # WL hash
16         buckets.setdefault(h, []).append(g)
17     return buckets

```

Listing 4: WL-hash-accelerated novelty and uniqueness computation (metrics.py).